UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

Facultad de Ciencias

Sistemas Operativos

Bootstrapping y Bare Metal en RP2040

Práctica 1

Presenta:
Rojo Peña Manuel Ianluck

Profesor:
Javier León Cotonieto

Ayudante laboratorio:
Adrián Martínez Manzo

Grupo: 7004, 2026-2

Fecha de entrega:
16 de febrero, 2026

Introducción

El objetivo de esta práctica es comprender el proceso de *Bootstrapping* y la manipulación de hardware a bajo nivel (*Bare Metal*) en la arquitectura RP2040 de la placa Raspberry Pi Pico. Buscamos controlar el hardware utilizanod apuntadores y escribiendo directamente en direcciones físicas de memoria, sin apoyarnos del uso de librerías de abstracción como `pico_stdlib` o funciones de `sleep_ms`.

Adicionalmente, el programa se realizó en lenguaje C, como un códgio base de apoyo del aydaunte, junto con una configuró del entorno de desarrollo basado en *Cross-Compilation* utilizando `CMakeLists.txt`, componente crucial, el cual gestiona el ensamblaje del binario y añade el *Boot Stage 2*, necesario para la inicialización correcta de la placa.

Planteamiento del Problema

El reto consiste en visualizar un número de cuenta (el propio) utilizando la salida `GPIO` de la placa. El requerimiento original en las instrucciones indica el uso de 4 LEDs externos para representar cada dígito en paralelo (4 bits simultáneos). Sin embargo, debido a la carencia de componentes externos (LEDs, resistencias. protoboards, etc.), nos limitaremos únicamente al LED integrado en la placa (`GPIO 25`).

Para resolver esto, se diseñó e implementó un algoritmo que divide cada dígito del número de cuenta en 4 bits, manipulando bit a bit para ser mostrado en el LED inegrado. De este modo, en lugar de mostrar los 4 bits de un dígito espacialmente, se muestran temporalmente, iterando bit a bit con tiempos de encendido/apagado designados.

1. Bootstrapping y Configuración de Hardware

Por defecto, el hardware se encuentra en estado de *reset* para ahorrar energía. Accedemos a la dirección base del controlador de resets (`0x4000C000`) para habilitar los bancos de entrada/salida:

- `RESETS_RESET`: Limpiamos los bits correspondientes a `IO_BANK0` y `PADS_BANK0` para sacarlos del estado de reset.
- `RESETS_DONE`: Implementamos una espera activa hasta que el hardware confirme que los periféricos están listos.

Una vez activos, se configura el `GPIO 25` (nuestro querido LED integrado):

- `GPIO25_CTRL`: Escribimos un 5 para seleccionar la función `SIO` (*Software Control*), permitiendo que el núcleo manipule el pin directamente.
- `SIO_GPIO_OE`: Escribimos un 1 en la posición 25 para habilitar el pin como salida (*Output Enable*).

Esta seccion fue dada por el ayudante, y su respectivo código queda como sigue:

```c
#include <stdint.h>

// === MAPA DE MEMORIA (RP2040 Datasheet) ===
// 1. SUBSYSTEM RESETS (Base: 0x4000C000)
// Controla que perifericos estan despiertos o dormidos .
#define RESETS_BASE 0x4000C000
#define RESETS_RESET (*(volatile uint32_t *)(RESETS_BASE + 0x0))
#define RESETS_DONE (*(volatile uint32_t *)(RESETS_BASE + 0x8))

// 2. IO BANK 0 (Base: 0 x40014000)
// Controla la funcion logica de los pines (GPIO , UART , etc).
#define IO_BANK0_BASE 0x40014000
// El pin 25 tiene un offset de 0x0CC
#define GPIO25_CTRL (*(volatile uint32_t *)(IO_BANK0_BASE + 0x0CC))

// 3. SINGLE - CYCLE IO (SIO) (Base: 0xD0000000)
// Controla el estado alto / bajo de los pines de forma rapida.
#define SIO_BASE 0xD0000000
#define SIO_GPIO_OE (*(volatile uint32_t *)(SIO_BASE + 0x020)) //
    Output Enable
#define SIO_GPIO_OUT (*(volatile uint32_t *)(SIO_BASE + 0x010)) //
    Output Value

// NOTA: Si usas Pico W, cambia 25 por un pin externo (ej . 16)
#define LED_PIN 25
```

Código 1: Definición de Registros y Mapeo de Memoria, main.c

Por otro lado, el proceso de inicialización del microcontrolador RP2040 se realiza en dos etapas: el levantamiento de los subsistemas desde el controlador de Resets y la configuración de la multiplexación de pines GPIO.

```
// --- PASO 1: BOOTSTRAPPING (Despertar Hardware) ---
// Debemos quitar el Reset a dos componentes:
// Bit 5: IO_BANK0 (Logica de pines)
// Bit 8: PADS_BANK0 (Electricidad de pines)
// Escribimos 0 en esos bits para sacarlos del reset.
RESETS_RESET &= ~((1 << 5) | (1 << 8) );
// Esperamos hasta que el hardware confirme que desperto
while((RESETS_DONE & ((1 << 5) | (1 << 8))) != ((1 << 5) | (1 << 8))
 );

// --- PASO 2: CONFIGURACION ---
// Seleccionamos la Funcion 5 (SIO - Software Control) para el Pin
 25
GPIO25_CTRL = 5;
// Habilitamos el Pin 25 como SALIDA en el SIO
SIO_GPIO_OE |= (1 << LED_PIN);
```

Código 2: Rutina de Bootstrapping y Configuración de GPIO.

A diferencia de arquitecturas convencionales, el RP2040 mantiene sus periféricos apagados para ahorrar energía. Para habilitarlos, se debe escribir un 0 en el bit correspondiente del registro de resets.

Se utiliza la operación `&= m̃ask` para forzar los bits 5 (IO_BANK0) y 8 (PADS_BANK0) a cero sin alterar el estado de otros periféricos. Con IO_BANK0 habilitamos la lógica de los pines 0-29, mientras que con PADS_BANK0 habilitamos las características eléctricas (voltaje, impedancia) de los pines.

La activación del hardware no es instantánea, necesitamos implementar un ciclo de espera (conocido como *polling*) verificando el registro RESETS_DONE. El sistema se detiene hasta que la operación lógica AND confirma que ambos bits (5 y 8) han cambiado a 1, indicando que los subsistemas están estables.

Cada pin en el RP2040 puede tener múltiples funciones, pero al escribir el valor 5 en el registro de control del pin 25, conectamos físicamente el pin al bloque **SIO** (Single-cycle IO), permitiendo al procesador controlarlo directamente por software.

Finalmente, una vez que el pin es controlado por el **SIO**, debe configurarse como salida. Esto lo logramos escribiendo un 1 en la posición 25 del registro *Output Enable* (OE). Se utiliza el operador OR (|=) para no afectar la dirección de otros pines que pudieran estar configurados.

2. Algoritmo para representación binaria

Para la visualización del número de cuenta, implementamos un algoritmo que descompone cada dígito decimal en sus 4 bits correspondientes en binario:

1. **Secuencia de Sincronización:** Tres parpadeos rápidos iniciales indican al usuario el inicio de la transmisión del número completo.
2. **Iteración de Dígitos:** Se recorre el arreglo `no_cuenta`.
3. **Descomposición de Bits:** Para cada dígito, se itera del bit menos significativo (bit 0) al bit 3.
4. **Estado del LED:** Si el bit es 1, el LED se mantiene encendido por un tiempo `BIT_ON`. Si es 0, se apaga durante ese tiempo.
5. **Separadores:** Utilizamos tiempos muertos (`BIT_OFF` y `DIGIT_DELAY`) para distinguir entre bits individuales y entre cambios de dígito.

Implementación del Algoritmo:

De este modo el algoritmo queda como sigue:

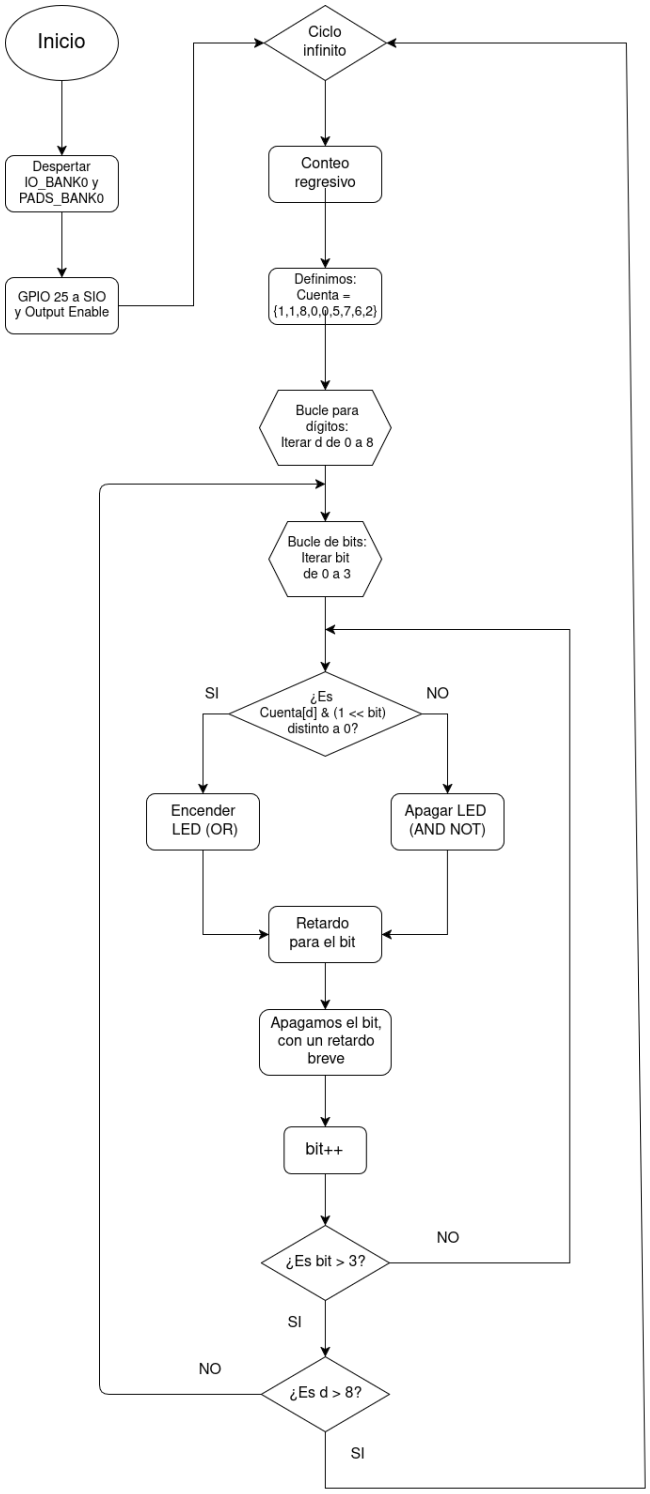
```c
//valores para los tiempos del LED prendido/apagado
const int START_LED_ON = 15000000;
const int START_LED_OFF = 5000000;
const int BIT_ON = 8000000;
const int BIT_OFF = 2500000;
const int DIGIT_DELAY = 6000000;

/* Funcion auxiliar para dar un retraso (o delay) forzado entre las acciones de prender y apagar el LED */
int delay(int time) {
  for (volatile int i = 0; i <= time; i ++);
}

/*Funcion principal para prender nuestro numero de cuenta en el LED*/
int main() {
  // --- PASO 1: BOOTSTRAPPING (Despertar Hardware) ---
  // Debemos quitar el Reset a dos componentes:
  // Bit 5: IO_BANK0 (Logica de pines)
  // Bit 8: PADS_BANK0 (Electricidad de pines)
  // Escribimos 0 en esos bits para sacarlos del reset.
  RESETS_RESET &= ~((1 << 5) | (1 << 8) );
  // Esperamos hasta que el hardware confirme que desperto
  while((RESETS_DONE & ((1 << 5) | (1 << 8))) != ((1 << 5) | (1 << 8)) );

  // --- PASO 2: CONFIGURACION ---
  // Seleccionamos la Funcion 5 (SIO - Software Control) para el Pin 25
```

```
  GPIO25_CTRL = 5;
  // Habilitamos el Pin 25 como SALIDA en el SIO
  SIO_GPIO_OE |= (1 << LED_PIN);

  //el numero de cuenta a mostrar
  int no_cuenta[] = {1, 1, 8, 0, 0, 5, 7, 6, 2};

  // --- PASO 3: BUCLE INFINITO (Kernel Loop) ---
  while (1) {
    //damos un conteo regresivo para indicar que
    //se empezara a mostrar el numero de cuenta
    for (int i = 0; i < 3; i++) {
      SIO_GPIO_OUT |= (1 << LED_PIN); //prendemos el LED
      delay(START_LED_ON); //hacemos tiempo para que se note el conteo regresivo
      SIO_GPIO_OUT &= ~(1 << LED_PIN); //lo apagamos
      delay(START_LED_OFF); //damos otro tiempo de espera mas entre el conteo del LED
    }
    //iteramos digito por digito en el numero de cuenta
    for (int d = 0; d < 9; d++) {
      //iterador para prender 4 bits y representar el digito en binario
      for (int bit = 0; bit < 4; bit++) {
    //utilizamos |= y &= ~ para apagar y/o prender los LEDS dependiendo del estado del bit
    if (no_cuenta[d] & (1 << bit))
      SIO_GPIO_OUT |= (1 << LED_PIN); //si en la posicion del bit es 1, prendemos el LED
    else
      SIO_GPIO_OUT &= ~(1 << LED_PIN); //en otro caso lo apagamos

    //dejamos el bit/led prendido o apagado por
    delay(BIT_ON);
    //apagamos el LED entre bits
    SIO_GPIO_OUT &= ~(1 << LED_PIN);
    //damos una espera antes de continuar con el siguiente bit
    delay(BIT_OFF);
      }
      //damos un intervalo entre digitos
      //como ya apagamos el bit en el 4 bit del for anterior, no es necesario volverlo a apagar
      delay(DIGIT_DELAY);
    }
  }

  return 0;
}
```

Código 3: Algoritmo principal para visualización del número de cuenta en binario.

Diagrama de flujo del algoritmo

El diagrama de flujo del algoritmo queda como sigue:



3. Análisis de Componentes Críticos

1. Manipulación de Bits (Bitwise Operations)

La lógica central para determinar si el LED debe encenderse reside en la instrucción:

```
if (no_cuenta[d] & (1 << bit))
```

Esta operación es una **optimización computacional** frente al uso de divisiones o módulos aritméticos.

- **Máscara (Masking):** La expresión `1 « bit` genera un valor donde solo el bit en la posición `bit` es 1. Por ejemplo si `bit = 2`, genera `0100`).
- **Extracción (AND):** Al realizar la operación `AND (&)` con el número de cuenta, el resultado será diferente de cero *únicamente* si el número tiene un 1 en esa posición específica.

Ejemplo práctico: Para el dígito 5, en binario es `0101`:

- `bit = 0` (Máscara `0001`): 0101 & 0001 = 0001 (Verdadero → BIT_ON).
- `bit = 1` (Máscara `0010`): 0101 & 0010 = 0000 (Falso → BIT_OFF).
- `bit = 2` (Máscara `0100`): 0101 & 0100 = 0100 (Verdadero → BIT_ON).
- `bit = 3` (Máscara `1000`): 0101 & 1000 = 0000 (Falso → BIT_OFF).

2. Control de Registros

Para modificar el estado del LED sin afectar otros pines del `GPIO`, se utilizan operaciones de lectura-modificación-escritura a nivel de bits:

- **Encender (Set):** `SIO_GPIO_OUT |= (1 « LED_PIN);` Aquí se utiliza el operador **OR** inclusivo. Esto fuerza el bit 25 a 1, sin alter los bits de otros pines (como el 16, 17 si los usáramos), pues para nuestra implementación es innecesario operar sobre más pines.
- **Apagar (Clear):** `SIO_GPIO_OUT &= (1 « LED_PIN);` Primero se genera la máscara inversa con **NOT** (`~`). Si la máscara era `0100`, la inversa es `1011`. Al hacer **AND**, forzamos a 0 solo el bit que nos interesa, preservando el estado de los demás.

3. Implementación de Retardos

Dado que no disponemos de interrupciones de reloj ni funciones del sistema operativo, necesitamos implementar una forma de mantener ocupada:

```
int
delay(int time)
{
  for (volatile int i = 0; i <= time; i++);
}
```

Se decidió usar una función auxiliar para manejar los retardos de modo que sea más fácil leer y entender el algoritmo, pues ahora solo se necesita llamar a la función `delay()` con el tiempo que se desea aplicar al retardo como parámtro.

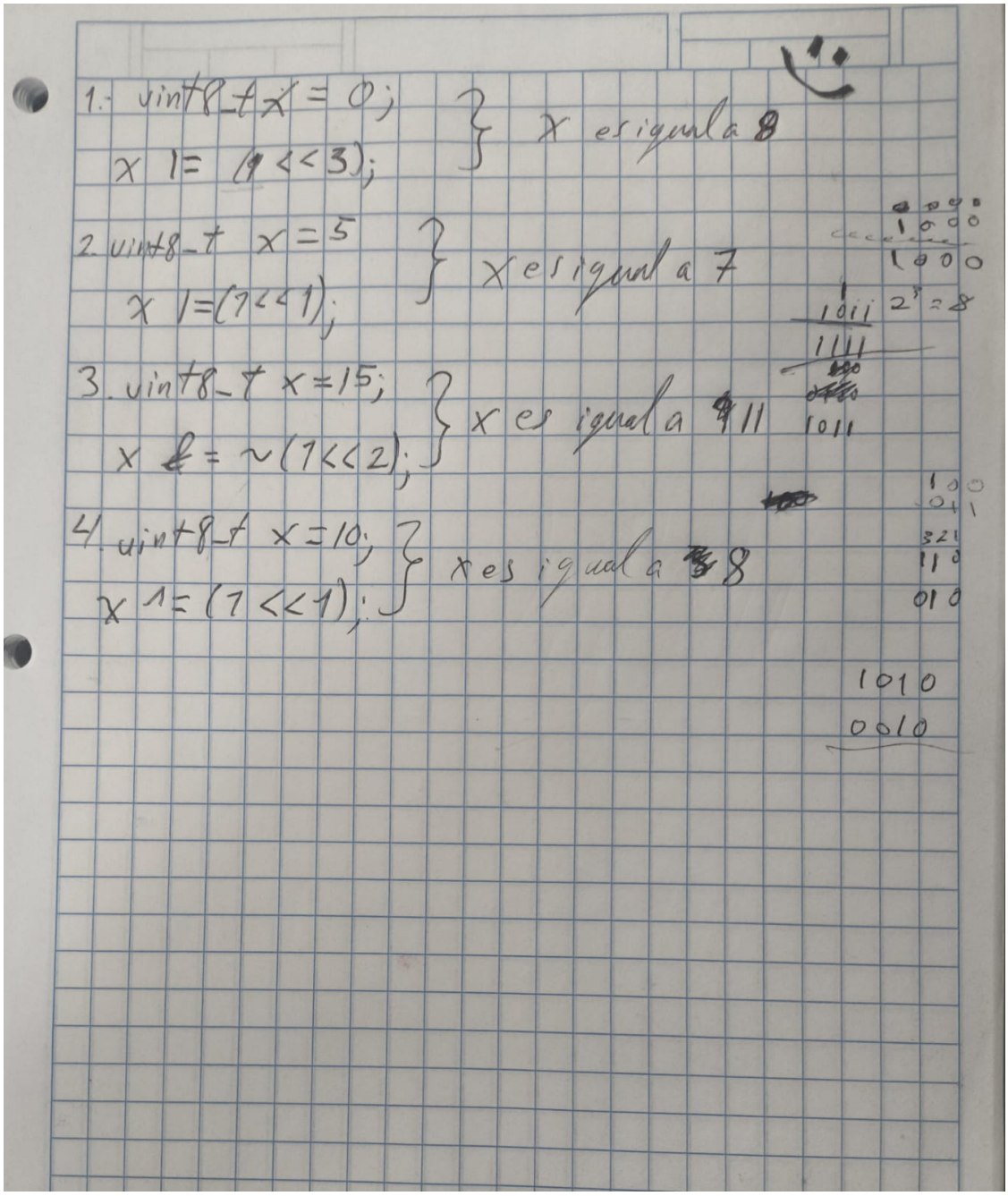
El uso del calificador`volatile` es muy necesario, es crítico. Sin él, el compilador GCC con banderas de optimización activadas detectaría que el bucle no tiene efectos secundarios y eliminaría el código por completo, haciendo que el retraso fuera de 0 segundos. Con `volatile` forzamos a la CPU a realizar los ciclos de lectura/escritura en memoria, consumiendo ciclos de reloj y generando el retardo necesario.

Evidencias

A continuación se presentan las pruebas de funcionamiento. El parpadeo sigue la secuencia binaria esperada para el número de cuenta:

Evidencia Pi Pico con #Cuenta en Binario

Actividad en clase para validación



1. uint8_t x = 0;
   x |= (1<<3);
   } x es igual a 8

2. uint8_t x = 5
   x |=(1<<1);
   } x es igual a 7

3. uint8_t x=15;
   x &= ~(1<<2);
   } x es igual a 11

4. uint8_t x=10;
   x ^= (1<<1);
   } x es igual a 8

1000
1000
1011 $2^3=8$
1111
1011

1010
0010

Concluciones

1. **Viabilidad del Bare Metal:** Mostramos que es posible controlar hardware como el `RP2040` sin depender de toda la implementación que viene en el SDK estándar, manipulando directamente el mapa de memoria. Esto reduce significativamente el tamaño del binario final y nos otorga un mejor control total sobre los ciclos de CPU.

2. **Optimización mediante Bitwise:** Con la implementación de nuestro algoritmo de visualización se evidenció que el uso de operadores a nivel de bits (`«`, `&`, `|`, `~`) es más eficiente y práctico en términos de instrucciones de ensamblador que las operaciones aritméticas tradicionales para la conversión de bases numéricas.

3. **Solución a Restricciones de Hardware:** La multiplexación temporal implementada permitió cumplir con el requerimiento de visualizar datos de 4 bits utilizando un canal de salida de 1 bit (el LED integrado). Aunque la tasa de transferencia de información es menor (visualización más lenta), se logró transmitir la información completa sin hardware adicional.

4. **Importancia de Volatile:** Se verificó experimentalmente que, en programación de sistemas embebidos, el control sobre la optimización del compilador es vital para tareas de temporización por software (*software delays*).